

Claude + Codex Cheatsheet

Quick reference for the commands I always forget. Open anytime with: `open ~/Documents/GitHub/bexperiments/CHEATSHEET.md` (or just ask Claude "show my cheatsheet").

The mental model: project, folder, working files, output

Term	What it means	Example
Project	The durable container for one continuing outcome. In Claude/Codex it may also group chats and instructions.	"Littler AI Strategy"
Project folder	The actual Finder folder containing the project's files. An app project does not always create one automatically.	<code>~/Documents/Projects/Littler AI Strategy/</code>
Working files	Drafts, analysis, downloads, renders, and temporary material used to make the deliverable.	<code>working/</code> , <code>work/</code> , scratch files
Output	A finished deliverable someone will read, use, send, or publish. It normally belongs inside its project.	<code>outputs/Strategy Deck.pptx</code>
Knowledge	Notes, decisions, tasks, status, and ideas worth remembering across sessions.	Obsidian Second Brain

Golden rule: organize by the real-world project, not by which AI made the file. Claude and Codex are tools working on Rebecca's projects; they should not create separate universes of projects.

Where Rebecca's files currently land

Tool / kind of work	Where it lands
Obsidian knowledge	<code>~/Library/Mobile Documents/iCloud-md-obsidian/Documents/Second Brain/</code> - the canonical vault
Claude Cowork project	The project exists in Claude; only files written to an attached local folder become normal Finder files
Claude Code	Whatever folder Claude was launched from. The canonical repo is <code>~/Documents/GitHub/bexperiments/</code>
Codex local project	Its attached primary folder ; new chats start there
Codex projectless chat	Hidden runtime scratch at <code>~/Documents/Codex/YYYY-MM-DD/generated-name/</code> ; generated names are not real projects
Code	Canonical home: <code>~/Documents/GitHub/</code>
One-off, no project yet	Landing pad: <code>~/Documents/AI Inbox/YYYY-MM/</code>

Codex Desktop requirement: `~/Documents/Codex` must be a real, writable directory. Do not replace it with a symlink, even one pointing to another valid folder. If Work or Codex shows "Projectless thread directory must be a real directory," run `if [ -L ~/Documents/Codex ]; then unlink ~/Documents/Codex; fi; mkdir -p ~/Documents/Codex`, fully quit the desktop app with **Command-Q**, and reopen it. Removing the symlink does not remove the folder or files it pointed to. Treat this directory as disposable staging. Move any finished keeper into its real project or the current **AI Inbox** month.

Before starting work, ask the agent: **"What folder are you working in, and where will the final output be saved?"** In Terminal, check for yourself:

```
pwd                # current working folder
open .             # show that folder in Finder
git rev-parse --show-toplevel  # root of the current git repo
```

For an ongoing project, attach or open its real folder first. Put final files in that project's `outputs/` folder. Put only project status, decisions, tasks, and durable notes in Obsidian.

Tokens, limits, and costs - three different things

Concept	What it controls	What happens when it fills
Context window	How much one chat can hold: prompts, replies, instructions, and files read	Older material is compacted/dropped, or start a fresh chat
Subscription usage limit	How much agent work the plan includes over a rolling period	Wait for reset, use a lighter model, upgrade, or add credits if offered
API billing	Pay-as-you-go use through an API key	Charges accumulate by input, cached-input, and output tokens; rate/spend limits still apply

A token is not a message. A short prompt can be expensive when the chat is already carrying a long history, large instructions, many files, tool results, or subagent work. Output/reasoning tokens usually consume allowance faster than cached input.

Why Claude may hit limits before Codex

- Claude, Claude Code, Cowork, desktop, and web usage can draw from the same Claude subscription allowance. Several simultaneous surfaces are not separate buckets.
- Long sessions repeatedly carry conversation history plus `CLAUDE.md` and files Claude has read. Opus, higher effort/extended thinking, tools, connectors, and subagents use more of the allowance.
- Codex has a separate OpenAI allowance. ChatGPT Work and Codex share OpenAI's agentic usage, but that pool is unrelated to the Claude pool.
- Not hitting Codex limits does **not** mean Codex is unlimited. It means the current OpenAI plan, model, workload, and included allowance have not reached the applicable limit.

Stretch Claude usage

Action	When to use it
<code>/usage</code>	Check plan usage and reset status
<code>/context</code>	See what is filling the current context window
<code>/clear</code>	New task or topic; strongest way to stop carrying old history
<code>/compact</code>	Same task, but the chat has become long
<code>/model</code>	Use Sonnet for most work; reserve Opus for genuinely hard planning/debugging
Refer to a file path	Lets Claude read selectively; avoid pasting giant files/logs into chat

Also keep `CLAUDE.md` concise, disable unused connectors/tools, avoid keeping unrelated tasks in one session, and do not leave multiple expensive sessions running unnecessarily.

What am I actually paying?

- **Subscription login:** the monthly plan includes an allowance. You are not normally shown a dollar charge for every prompt; you hit usage limits instead.
- **Claude Code with an API key:** pay-as-you-go. `/cost` shows session token usage/spend; check the Claude Console and any auto-reload setting.
- **Codex with a ChatGPT plan:** included usage is consumed first. Check **Codex Settings -> Usage** for remaining allowance/credits. Plus is currently \$20/month; Codex Pro tiers start at \$100/month and advertise 5x or 20x the Plus rate limits.
- **Codex with an API key:** pay only for the tokens used at the selected model's current API rates.

Pricing and plan limits change. Verify current details before buying or enabling auto-reload: [OpenAI Codex pricing](#), [Claude usage and length limits](#), and [Claude Code models, usage, and limits](#).

Terminal multiplexer (tmux workspace)

```
./tmux-bexperiments.sh # launch from ~/Documents/GitHub/bexperiments
```

- One window per project, Claude auto-started in each
- `tmux attach -t bexperiments` - reattach if already running
- `NO_CLAUDE=1 ./tmux-bexperiments.sh` - launch without auto-starting Claude
- `./tmux-bexperiments.sh some-folder` - also add a window for a new project folder

Moving around inside tmux

Keys	What it does
<code>Ctrl-b</code> then <code>w</code>	window list - pick with arrows + Enter
<code>Ctrl-b</code> then a number	jump straight to that window
<code>Ctrl-b</code> then <code>n / p</code>	next / previous window
<code>Ctrl-b</code> then <code>d</code>	detach (everything keeps running)

Claude Code

```
claude # start Claude in the current folder
```

- `claude --continue` (or `-c`) - resume the most recent session in this folder
- `claude --resume` - pick an older session to resume
- `claude "fix the failing test"` - start a session with an opening prompt
- `claude -p "explain this repo"` - one-shot answer, no interactive session
- `claude --model opus` - start with a specific model
- `claude update` - update Claude Code itself to the latest version

Launching Claude in bexperiments

Remember: it's always two steps - `cd` into `bexperiments` first, THEN launch `claude`. Claude works in whatever folder you start it from, so if you skip the `cd` it won't see the repo. As one line:

```
cd ~/Documents/GitHub/bexperiments && claude
```

Shortcut from any folder: `bex-claude`

```
cd ~/Documents/GitHub/bexperiments/<project> && claude
```

- **Repo-wide session** (first command): sees all projects at once
- **One project only** (second command): keeps Claude focused on that folder
- **Skip both steps entirely:** `./tmux-bexperiments.sh` - every project window already has Claude running in the right folder; just switch windows and type

Slash commands (typed inside a Claude session)

Command	What it does
<code>/model</code>	switch models (Opus / Sonnet / Haiku) mid-session
<code>/clear</code>	wipe the conversation, start fresh (same folder)
<code>/compact</code>	summarize the conversation to free up context
<code>/resume</code>	switch to a different past session
<code>/cost</code>	show token usage / cost for the session
<code>/init</code>	generate or refresh the project's CLAUDE.md
<code>/memory</code>	view & edit what Claude remembers about this repo
<code>/permissions</code>	manage which tools/commands Claude may run without asking
<code>/mcp</code>	manage MCP server connections
<code>/config</code>	settings (theme, model default, etc.)
<code>/help</code>	list all commands

Keyboard shortcuts (inside a Claude session)

Keys	What it does
<code>Esc</code>	interrupt Claude mid-response
<code>Esc Esc</code>	jump back to an earlier message / rewind
<code>Shift+Tab</code>	cycle permission modes (normal -> auto-accept -> plan mode)
<code>!</code> at start of line	run a shell command directly (bash mode)
<code>@</code>	mention a file so it's pulled into context
<code>#</code> at start of line	save a note to memory (CLAUDE.md)
<code>Ctrl+C</code> twice	quit

Handy habits

- Start a big task in **plan mode** (`Shift+Tab` twice): Claude proposes a plan you approve before it touches files
- `#` notes are the cheap way to teach Claude repo conventions as you go
- Tell Claude "remember this" to save something across sessions

Codex (OpenAI / GPT)

Same two-step rhythm as Claude: `cd` into the folder first, then launch.

```
cd ~/Documents/GitHub/bexperiments && codex      # repo-wide Codex session
```

Shortcut from any folder: `bex-codex`

Use `bex` to move to the repository root, or `bex doc-find-replace` to move to one named subproject. Run `pwd` if you ever want to confirm where the next CLI agent will start.

- `codex "fix the failing test"` - start with an opening prompt
- `codex resume` - pick a previous session to resume
- `codex resume --last` - continue the most recent session (like `claude -c`)
- `codex -m <model>` - start with a specific GPT model, or type `/model` inside a session to switch
- `codex update` - update Codex itself to the latest version
- `codex login` / `codex logout` - manage the ChatGPT account it runs on
- `codex doctor` - troubleshoot if Codex is acting up (checks install, config, and auth)

- `codex exec "explain this repo"` - one-shot answer, no interactive session (like `claude -p`)

Claude <-> Codex quick translation

Task	Claude	Codex
Launch in current folder	<code>claude</code>	<code>codex</code>
Continue latest session	<code>claude -c</code>	<code>codex resume --last</code>
Pick an older session	<code>claude --resume</code>	<code>codex resume</code>
One-shot, no session	<code>claude -p "..."</code>	<code>codex exec "..."</code>
Switch model	<code>/model</code>	<code>/model</code>
Update the tool	<code>claude update</code>	<code>codex update</code>

Turning workflows into loops & repeatable skills

Works in both Claude and Codex - and yes, plain English is the whole trick.

Loops (do this thing over and over)

Just describe it: **"turn this into a loop"**, "keep doing this every 10 minutes", "re-run this check until the tests pass". Both tools understand.

- **Claude** also has an explicit command: `/loop 5m <prompt or /command>` (e.g. `/loop 10m check if the deploy finished`). Leave off the interval and Claude paces itself. Say "stop the loop" to end it.
- **Claude scheduled runs:** ask "schedule this to run every morning at 8" - that creates a recurring cloud agent that runs even when your laptop session is closed.
- **Codex:** just ask in plain language, or for something outside a session, wrap `codex exec "..."` in a shell loop or cron job.

Skills (save a workflow so you can rerun it by name)

When you've done a workflow once and want it on tap forever, tell the tool: **"turn what we just did into a repeatable skill called <name>".**

- **Claude:** creates a skill you invoke by typing `</name>` in any future session. Skills live in `.claude/skills/` (this repo only) or `~/.claude/skills/` (everywhere) - ask for whichever you want.
- **Codex:** same idea - its skills live in `~/.codex/skills/` and are invoked by name too.
- Good candidates: anything you've now explained to the agent twice.
- **Record a Skill (desktop app only):** in a Cowork task, click the + menu -> **Record a Skill**. Screen-record yourself doing the workflow while narrating the important choices, and Claude turns it into a skill - no typing out steps. ~10 min limit; needs Pro/Max/Team.

Cloning a repo (copying a project from GitHub)

Same two-step rhythm: clone it, then `cd` into the folder it creates and launch Claude.

```
git clone https://github.com/<user>/<repo>.git
cd <repo>
claude
```

- The URL is on the repo's GitHub page - green **Code** button -> HTTPS -> copy
- `git clone` makes a new folder named after the repo, right where you ran it
- Example - this whole monorepo onto a fresh machine: `git clone https://github.com/rbpasternak-tech/bexperiments.git`
- To grab someone else's repo, clone it into `~/Documents/GitHub/`, **not** inside `~/Documents/GitHub/bexperiments` - it is its own git repo, and nesting one repository inside another confuses git.
- Later, `git pull` from inside the folder gets the latest updates

New project flow

Use this flow only for code that belongs in the `bexperiments` monorepo. For a document, trip, application, or other non-code outcome, use a named folder under `~/Documents/Projects/` instead.

For a new `bexperiments` code project, create a descriptive folder and then relaunch the workspace script with that folder name:

```
mkdir ~/Documents/GitHub/bexperiments/contract-review-playground
./tmux-bexperiments.sh contract-review-playground
```

(Without tmux, the manual version is the same two-step idea: `cd ~/Documents/GitHub/bexperiments/contract-review-playground && claude`)

Then jump to its window (`Ctrl-b w`) and talk to Claude. If the project sticks, add it to the `PROJECTS` list in `tmux-bexperiments.sh`, the catalog in `README.md`, and the matching inventory in `CLAUDE.md`.

Output routing at a glance

Context	Temporary landing zone	Finished keeper
Second Brain knowledge	No temporary copy required	Obsidian <code>Second Brain</code> - only when Rebecca explicitly requests the edit
Generic Claude Cowork task	<code>~/Documents/AI Scratch/Claude Cowork/Claude Project Output/</code>	Named project <code>outputs/</code> , or <code>AI Inbox/YYYY-MM/</code> if projectless
OpenAI Work / Codex projectless task	Hidden <code>~/Documents/Codex/YYYY-MM-DD/generated-name/</code>	Named project <code>outputs/</code> , or <code>AI Inbox/YYYY-MM/</code> if projectless
Non-code project	Project-local <code>work/</code>	<code>~/Documents/Projects/<Project Name>/outputs/</code>
Code belonging to this monorepo	Project-local folder inside <code>bexperiments</code>	<code>~/Documents/GitHub/bexperiments/<project>/</code>
Separate codebase	Its own repository worktree	Its own named repo under <code>~/Documents/GitHub/</code>

`bexperiments` is not the catch-all destination for AI output. It is the home for code that belongs to that monorepo. Organize by the real outcome, not by whether Claude, Cowork, OpenAI Work, or Codex created it.